

操作系统

第二章 进程管理

Unix V6++的内存管理子系统

同济大学计算机系



Q1: 以下指令执行时，几次访存（访问内存）

- `inc %eax`
 - 1次访存。取这条指令。
 - 操作数`eax`是寄存器，在CPU内部。读写这个操作数没必要访问内存。
- `mov value, %eax`
 - 2次访存。1次取指，1次读内存变量`value`。
- `inc value`
 - 3次访存。1次取指，1次读内存变量`value`，1次写内存变量`value`。
- `push %ebp`
 - 2次访存。1次取指，1次写栈顶元素。



Q2、u变量是现运行进程的user结构，为什么

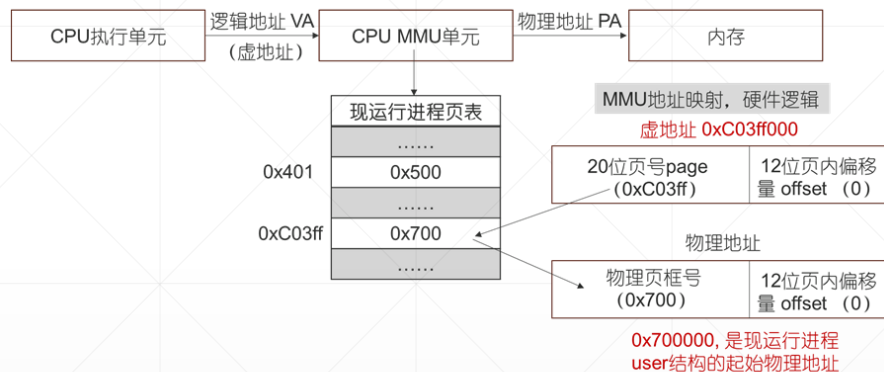
```
User& u = Kernel::Instance().GetUser();
```

```
User& Kernel::GetUser()  
{  
    return *(User*)USER_ADDRESS; // 0xC03ff000  
}
```

答：地址映射时，CPU查现运行进程页表

0	null	映射用户空间
1~0x400	 null	
0x401	0x500	
0x402	0x501	
0x403	0x701	
0x404	0x702	
0x405	null	
0x406~0x7fe	 null	
0x7ff	0x703	
	 null	
0xC0000	0	映射核心空间
0xC0001	1	
.....		
.....	0x3fe(1022)	
0xC03ff	0x700	
	 null	

(a)



(b)

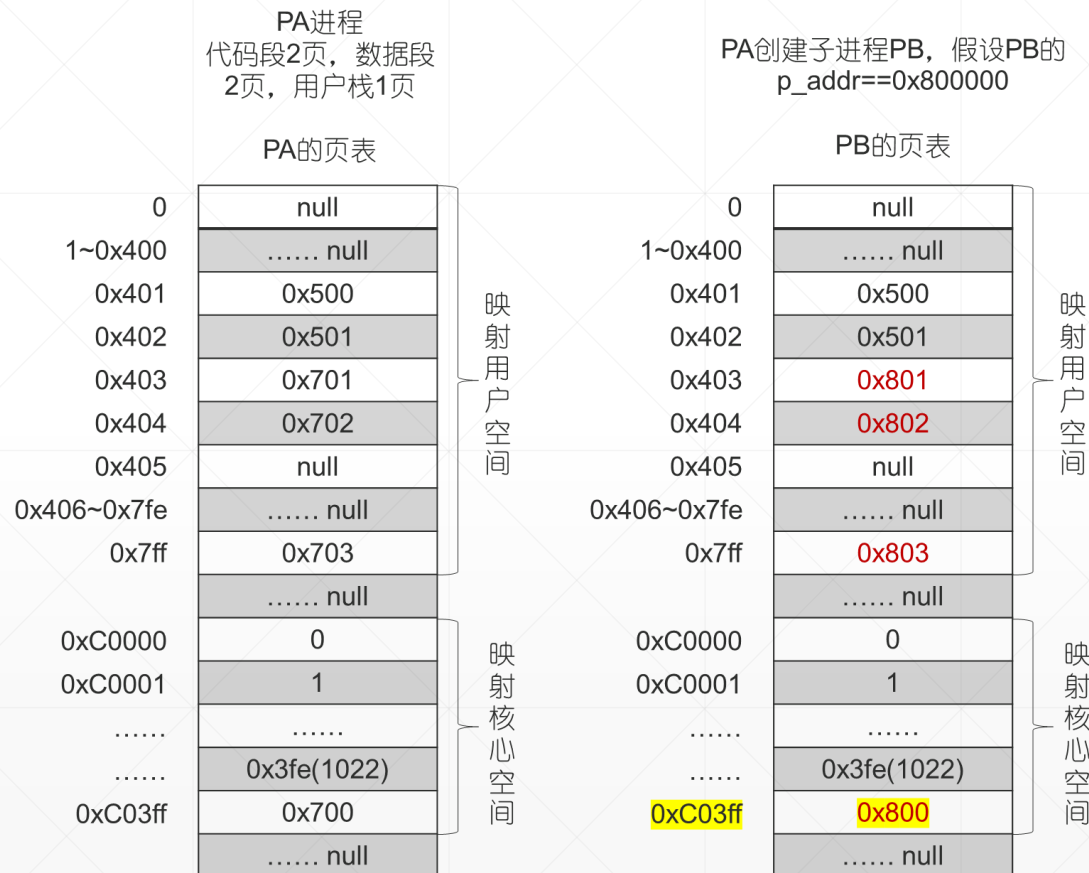
例：图 a 是PA进程的页表。代码段2页，数据段2页，用户栈1页。

可交换部分起始物理地址
 $p_addr == 0x700000$ ，
 共享正文段起始物理地址
 $x_caddr == 0x500000$ 。

PA是现运行进程时，CPU执行单元给出u变量的逻辑地址 0xC03ff000。

MMU地址映射后，得到物理地址 0x700000，PA user结构的首地址。

不同现运行进程，相同VA，引用的是不同的内存变量，属于不同进程。举例：



相同下标，红色的物理页框号分配给PB是它的数据段、堆栈段、PPDA

PB运行时，CPU执行单元给出虚地址0xC03ff000，得到的u变量是PB的user结构。

顺带观察页表：虚空间中，PA、PB如何共享正文段

PA进程
代码段2页，数据段
2页，用户栈1页

PA的页表

0	null	映射 用户 空间
1~0x400	 null	
0x401	0x500	
0x402	0x501	
0x403	0x701	
0x404	0x702	
0x405	null	
0x406~0x7fe	 null	
0x7ff	0x703	
	 null	
0xC0000	0	映射 核心 空间
0xC0001	1	
.....		
.....	0x3fe(1022)	
0xC03ff	0x700	
	 null	

PA创建子进程PB，假设PB的
p_addr==0x800000

PB的页表

0	null	映射 用户 空间
1~0x400	 null	
0x401	0x500	
0x402	0x501	
0x403	0x801	
0x404	0x802	
0x405	null	
0x406~0x7fe	 null	
0x7ff	0x803	
	 null	
0xC0000	0	映射 核心 空间
0xC0001	1	
.....		
.....	0x3fe(1022)	
0xC03ff	0x800	
	 null	

PA运行时，给出地址 $\in [0x401000, 0x403000)$ ，访问存放在0x500#、0x501#物理页框的代码段。

PB给出地址 $\in [0x401000, 0x403000)$ ，访问的是同样的物理页框。



目录

1、Unix V6++的内存管理子系统

- 地址映射和内存保护
 - i386 MMU的硬件逻辑
 - 系统页表。系统页目录、系统用户页表（2页）、系统核心页表（1页）
 - 进程的相对虚实地址映射表
 - 为新运行进程建立地址映射关系
- 内存分配和释放（for 物理内存的用户区）
- 案例



一、内存管理系统

- 内存管理，由CPU硬件和内核共同实施。
 - 使用的CPU硬件单元：MMU、中断单元、CPU内部cache
- 内存管理
 - 地址映射、内存保护 CPU实施，内核辅助（参与硬件，包括 MMU和中断单元）
 - 内存分配 内核实施
 - 内存扩展 内核实施，中断单元辅助（若系统支持虚拟存储器）

中断单元，暂跳过

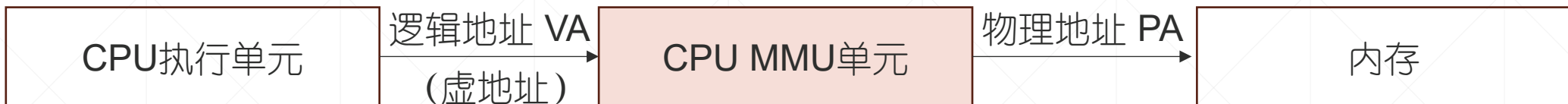


二、Unix V6++的内存管理系统，技术特征

- 页式
 - Intel i386, MMU支持二级页表。虚地址32位。
- 连续内存管理方式
 - 共享正文段、可交换部分连续存放。
- 内存扩充
 - 使用交换策略。内存紧张时, Unix V6++将部分进程图像放在盘交换区上。

三、内存管理（一） 地址映射 和 内存保护（主要由MMU实施）

▪ 地址映射



▪ 内存保护：禁止现运行进程非法内存访问，包括

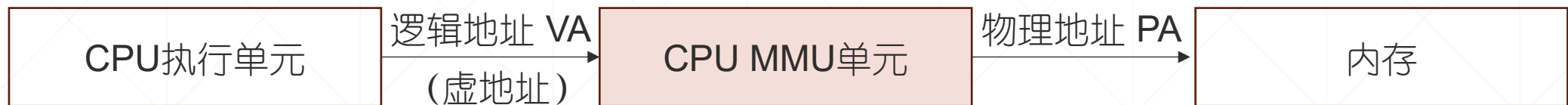
- 虚空间中不存在的代码/数据
- 内核
- 分配给其它进程的物理内存单元
- 写代码段、只读数据段
- 从数据段、只读数据段、bss段，堆栈段取指令

内核选中目标进程之后，为MMU提供新选中进程的虚空间布局和地址映射关系

随后，CPU执行这个新任现运行进程代码，循环以下操作，执行这个进程：

- 取指（下条指令）
- 译码执行
- 必要时访问内存，读写内存操作数（内存变量、常量）

地址映射 和 内存保护 (内核的辅助工作)

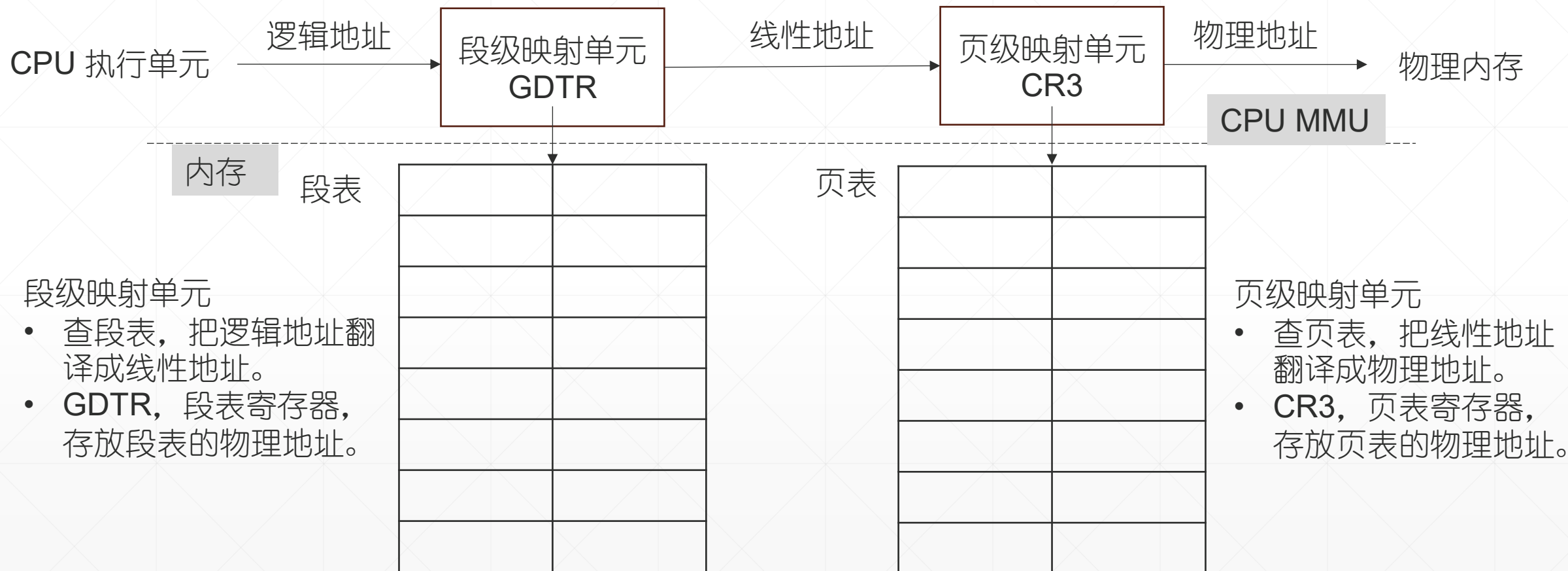


内核选中目标进程之后，为MMU提供新选中进程的虚空间布局和地址映射关系。

随后，CPU硬件执行这个新任现运行进程代码，循环以下操作，执行下面这个过程：

- 取指 (下条指令)
- 译码执行 (如果指令有内存操作数，访问内存，读写内存变量、常量)

案例：Unix V6++ 系统使用的MMU (i386段页式内存管理方式)

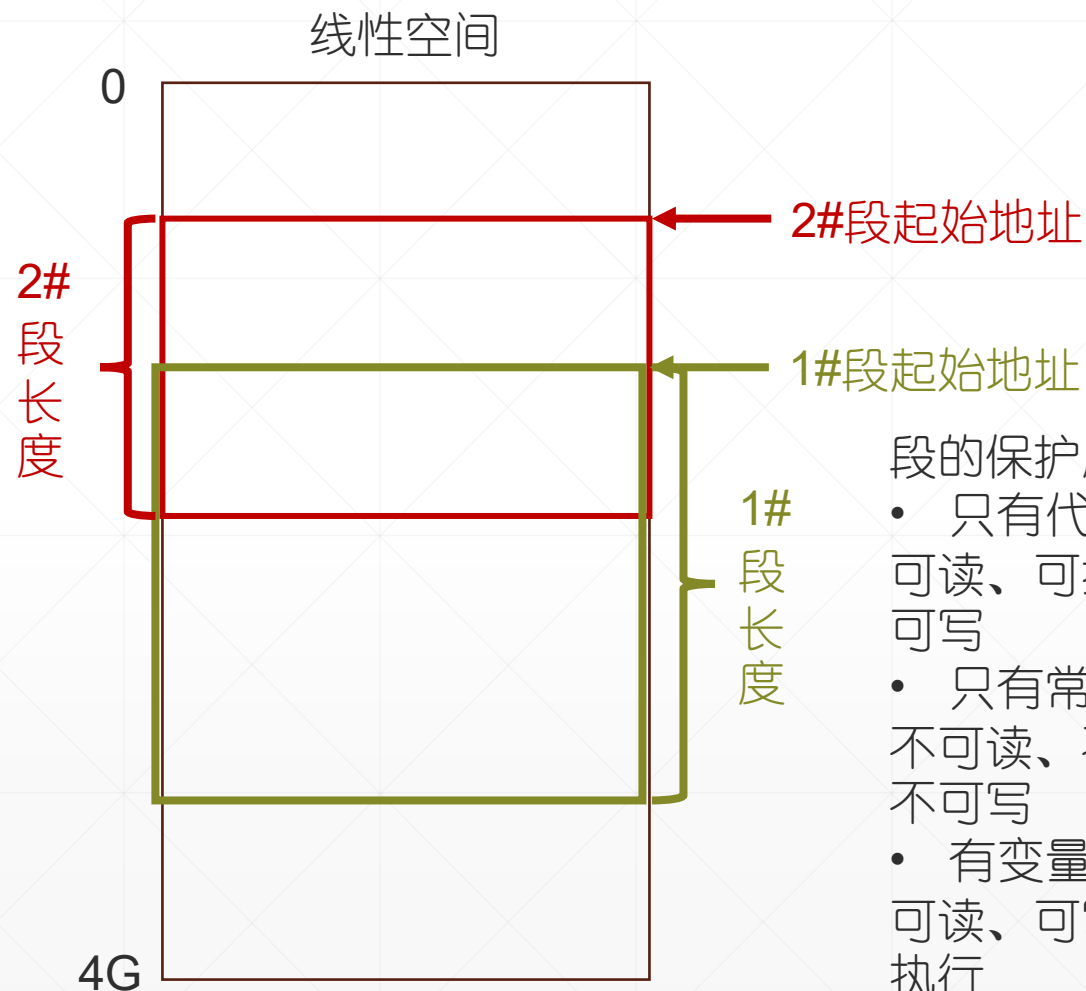


3.1 段级地址映射和内存保护



i386的段

- 进程的线性地址空间（虚空间），存放着有权访问的全部代码 & 数据。
 - Unix/Linux，一个应用程序 & 一个内核
- 线性地址空间分段。段是线性地址空间中的连续区域，存放代码 或 数据。
 - 一个段，可以是一个逻辑段，也可以包含多个逻辑段
 - 每个段，有唯一编号。
 - 允许段之间有重叠。



段的保护属性：

- 只有代码
可读、可执行、不可写
- 只有常量，
不可读、不可执行、不可写
- 有变量、没代码，
可读、可写、不可执行

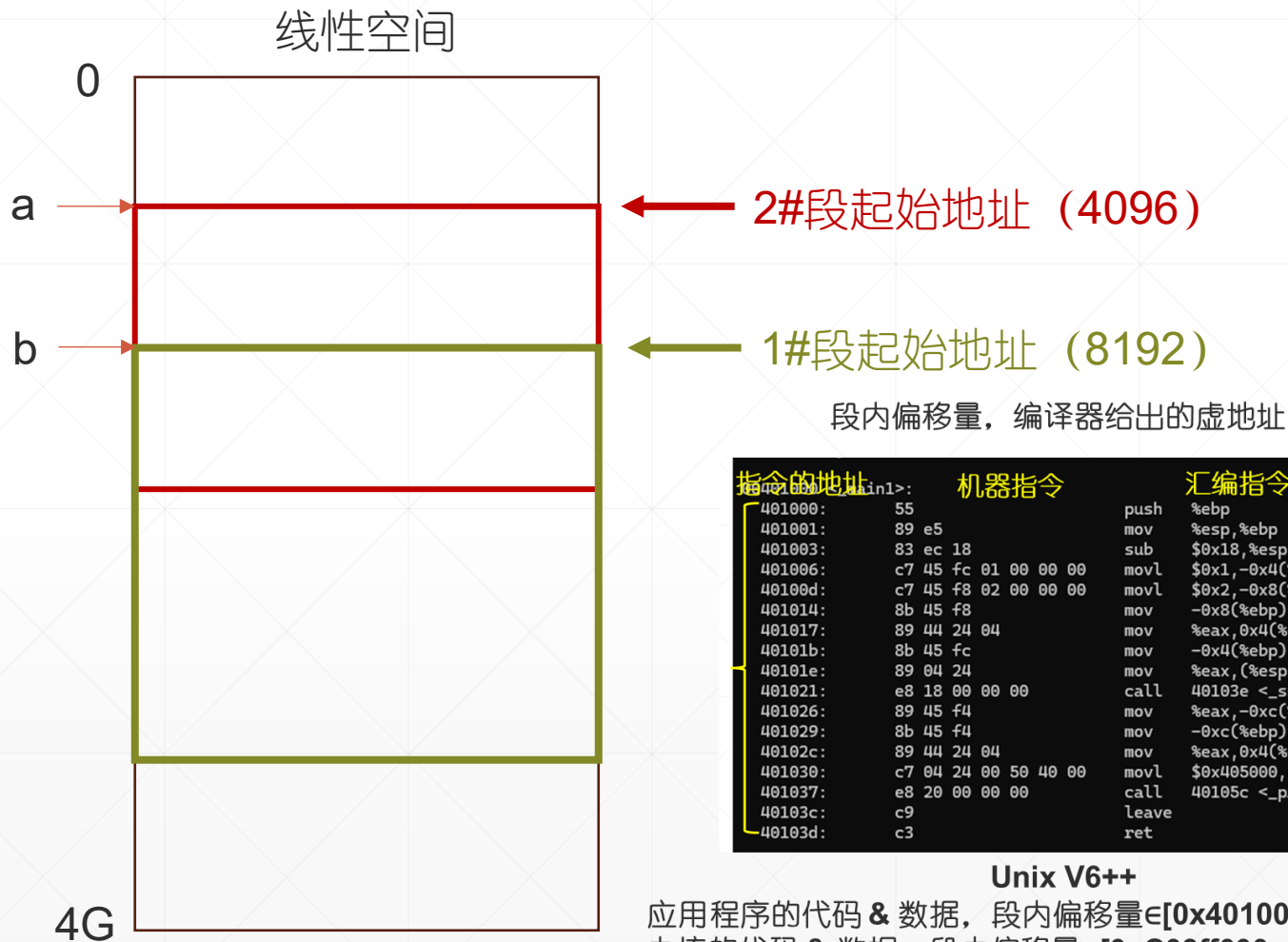
i386用2维逻辑地址引用线性空间中的单元

逻辑地址的格式:

段号(segID): 段内偏移量(offset)

变量	逻辑地址	线性地址
a	2:0	4096
b	2:4096 或 1:0	8192

地址映射 (逻辑地址 segID:offset) :
线性地址 = offset + 段segID的起始地址



Unix V6++使用的段 和 段表

段，描述进程虚空间（线性空间）中的连续区域

段表（每个表项，8字节）

进程虚空间包括4个段，所有进程皆如此

1#，用户代码段

2#，用户数据段

3#，核心代码段

4#，核心数据段

备注：0#段不存在，i386硬件不支持

5#，TSS段，不描述进程虚空间。这是一个内核数据结构，服务于i386中断元件，其中，esp0是核心栈，栈底的虚地址，Unix V6++，这个值是常数0xC0400000。

段号	起始偏移量 offset	段长 limit (减1)	段的保护属性 prot
0	null		
1	0	0xffffffff	S RO (核心代码段)
2	0	0xffffffff	S RW (核心数据段)
3	0	0xffffffff	U RO (用户代码段)
4	0	0xffffffff	U RW (用户数据段)
5	0	0xffffffff	S RW(核心数据)

i386, 段级 地址映射 和 内存保护

- CPU内部

段表寄存器: GDTR, 48位

段表起始物理地址 (32位)

段表长度(16位)

- 逻辑地址 LA:

段号 (segID) : 段内偏移量 (offset)

段表 (每个表项, 8字节)

段号	起始偏移量 offset	段长 limit (减1)	段的保护属性 prot
0	null		
1	0	0xffffffff	S RO (核心代码段)
2	0	0xffffffff	S RW (核心数据段)
3	0	0xffffffff	U RO (用户代码段)
4	0	0xffffffff	U RW (用户数据段)
5	0	0xffffffff	S RW(核心数据)

i386, 段级单元, 硬件逻辑

- 段号查段表, 得到2#段表项
- 内存保护
 - if(CPU用户态运行 & U/S为0)
 - 非法内存访问。应用程序无权访问内核, 立即终止现运行进程。
 - if(offset >= limit)
 - 非法内存访问。访问的单元不属于这个段, 终止现运行进程。
 - if(写内存单元 & R/W为0)
 - 非法内存访问。访问的单元不可写, 终止现运行进程。
 - if(CPU取指令 & 目标段X位是0, 不可执行)
 - 非法内存访问。修改代码段、只读数据段, 终止现运行进程。
- 地址映射
 - 线性地址 = 逻辑地址的offset + 目标段的起始偏移量



段表 (每个表项, 8字节)

段号	起始偏移量 offset	段长 limit (减1)	段的保护属性 prot
0	null		
1	0	0xffffffff	S RO (核心代码段)
2	0	0xffffffff	S RW (核心数据段)
3	0	0xffffffff	U RO (用户代码段)
4	0	0xffffffff	U RW (用户数据段)
5	0	0xffffffff	S RW (核心数据)

segID == 2

prot		U/S	R/W	
------	--	-----	-----	--

1, U 用户空间的段 1, W可写
0, S 核心空间的段 0, R只读

i386的术语 (1)

- 段表, 全局描述符表GDT (Global Descriptor Table)
- 段表中的表项, 段描述符 (每个描述符8字节)
- 段表寄存器, GDTR



段表 (每个表项, 8字节)

段号	起始偏移量 offset	段长 limit (减1)	段的保护属性 prot
0	null		
1	0	0xffffffff	S RO (核心代码段)
2	0	0xffffffff	S RW (核心数据段)
3	0	0xffffffff	U RO (用户代码段)
4	0	0xffffffff	U RW (用户数据段)
5	0	0xffffffff	S RW (核心数据)

i386的段寄存器

- CS, 代码段寄存器
- DS, 数据段寄存器
- SS, 堆栈段寄存器
- ES, 第2数据段寄存器
- FS, 第3数据段寄存器
- GS, 第4数据段寄存器



GDT

段表 (每个表项, 8字节)

段号	起始偏移量 offset	段长 limit (减1)	段的保护属性 prot
0	null		
1	0	0xffffffff	S RO (核心代码段)
2	0	0xffffffff	S RW (核心数据段)
3	0	0xffffffff	U RO (用户代码段)
4	0	0xffffffff	U RW (用户数据段)
5	0	0xffffffff	S RW(核心数据)

CPU的运行态

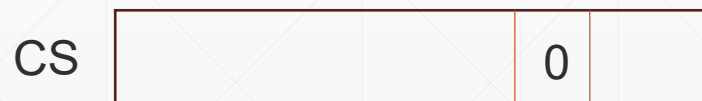
▪ 核心态运行时

- CS 0x8
- DS 0x10
- SS 0x10
- ES~其余寄存器
0x10

▪ 用户态运行时

- CS 0x1b
- DS 0x23
- SS 0x23
- ES~其余寄存器
0x23

CS的 0#、1# bit, 是CPL, 标识CPU的运行状态



CPL, Current Privilege Level

00, 正在执行内核代码 (核心态) // ring 0

11, 正在执行应用程序 (用户态) // ring 3

MMU段级映射，完整流程



段表 (每个表项, 8字节)

段号	起始偏移量 offset	段长 limit (减1)	段的保护属性 prot
0	null		
1	0	0xffffffff	S RO (核心代码段)
2	0	0xffffffff	S RW (核心数据段)
3	0	0xffffffff	U RO (用户代码段)
4	0	0xffffffff	U RW (用户数据段)
5	0	0xffffffff	S RW (核心数据)

段寄存器 (段描述子)



segID, 来自段寄存器中的段号, 段级单元用它查段表, 定位段描述符

offset, 来自CPU执行单元

- 段号查段表, 得到2#段表项
- 内存保护
 - if(CPU用户态运行 & U/S为0)
 - 非法内存访问。应用程序无权访问内核, 立即终止现运行进程。
 - if(offset >= limit)
 - 非法内存访问。访问的单元不属于这个段, 终止现运行进程。
 - if(写内存单元 & R/W为0)
 - 非法内存访问。访问的单元不可写, 终止现运行进程。
 - if(CPU取指令 & 目标段X位是0, 不可执行)
 - 非法内存访问。修改代码段、只读数据段, 终止现运行进程。
- 地址映射
 - 线性地址 = 逻辑地址的offset + 目标段的起始偏移量

例1：现运行进程用户态运行时，取代码段第一条指令

逻辑地址

CS:EIP

CS, 0x1b。展开成2进制, 0000000000011011。段号segID是3。
offset = EIP = 0x401000。



段级映射使用3#段表项

线性地址 = offset + 起始偏移量 0
= offset

段表 (每个表项, 8字节)

段号	起始偏移量 offset	段长 limit (减1)	段的保护属性 prot
0	null		
1	0	0xffffffff	S RO (核心代码段)
2	0	0xffffffff	S RW (核心数据段)
3	0	0xffffffff	U RO (用户代码段)
4	0	0xffffffff	U RW (用户数据段)
5	0	0xffffffff	S RW (核心数据)

例 2：现运行进程核心态运行时，访问内存操作数

1、压栈操作：push %eax
逻辑地址 SS : ESP-4

2、mov %eax, value
逻辑地址 DS : value地址

3、mov -4%ebp, %eax
逻辑地址 SS : EBP-4



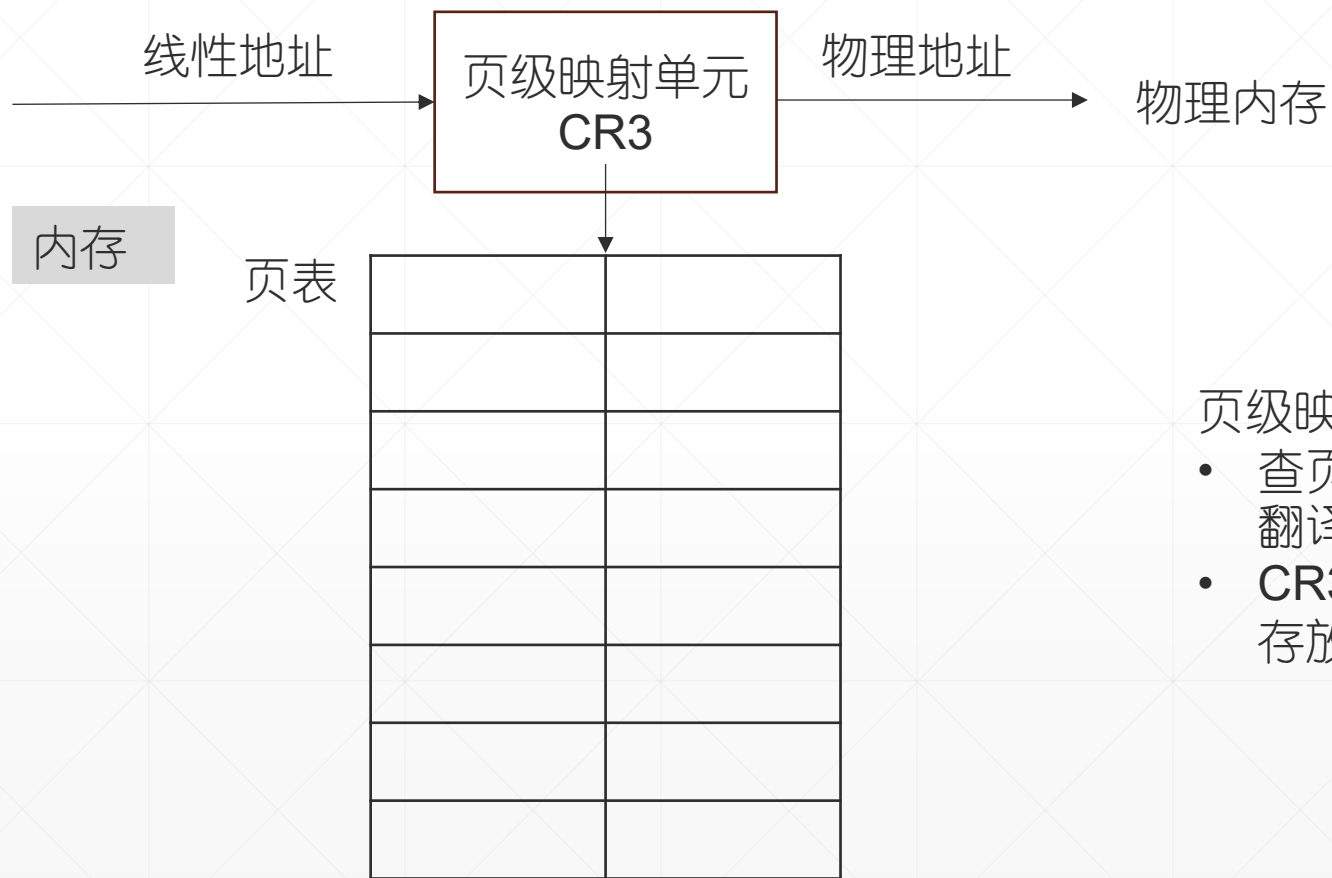
段表（每个表项，8字节）

段号	起始偏移量 offset	段长 limit (减1)	段的保护属性 prot
0	null		
1	0	0xffffffff	S RO (核心代码段)
2	0	0xffffffff	S RW (核心数据段)
3	0	0xffffffff	U RO (用户代码段)
4	0	0xffffffff	U RW (用户数据段)
5	0	0xffffffff	S RW(核心数据)

无论何种情况，使用**2#**段描述子映射

线性地址 = offset

3.2 页级 地址映射 和 内存保护



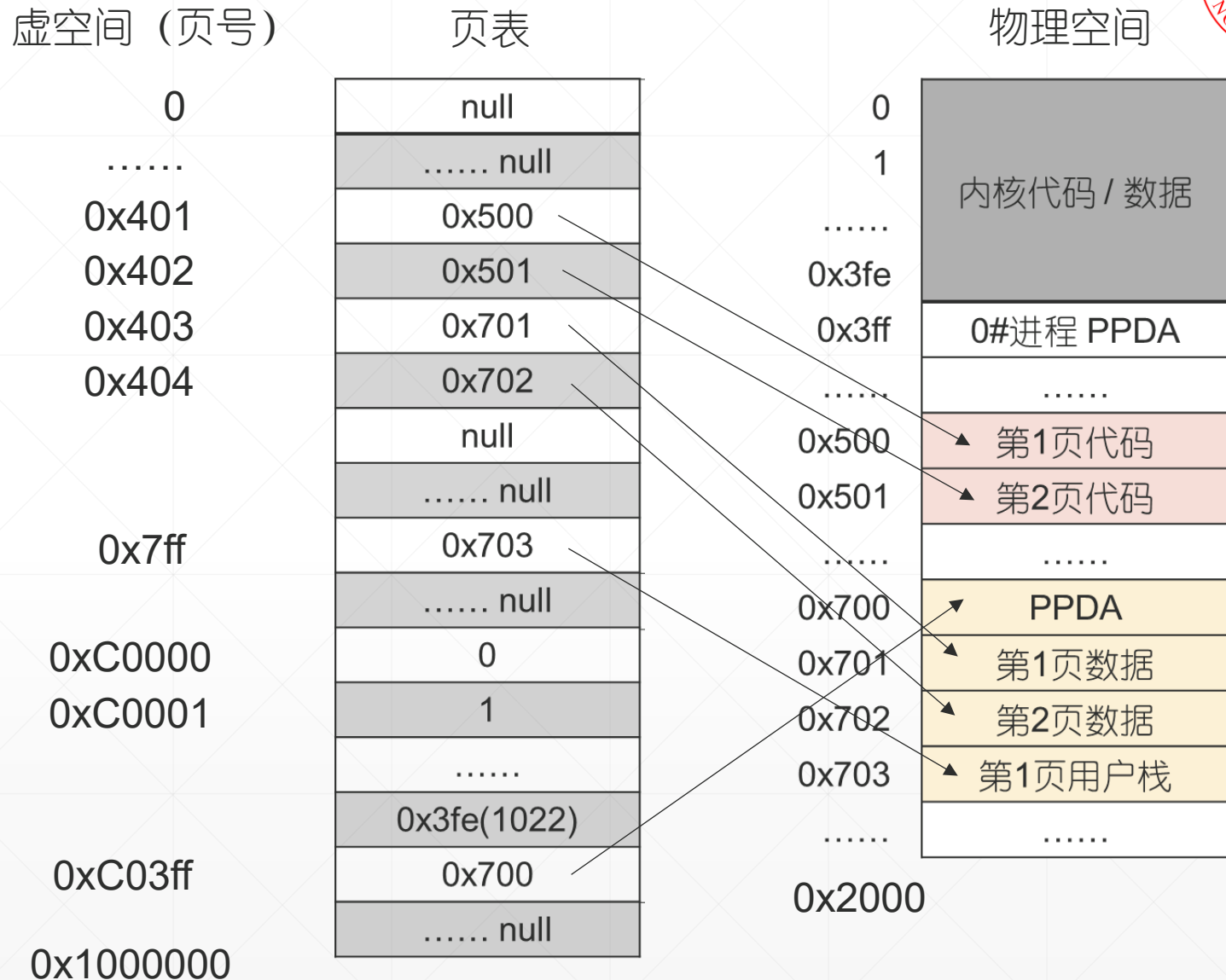
页级映射单元

- 查页表，把线性地址翻译成物理地址。
- **CR3**，页表寄存器，存放页表的物理地址。

页式MMU的优点

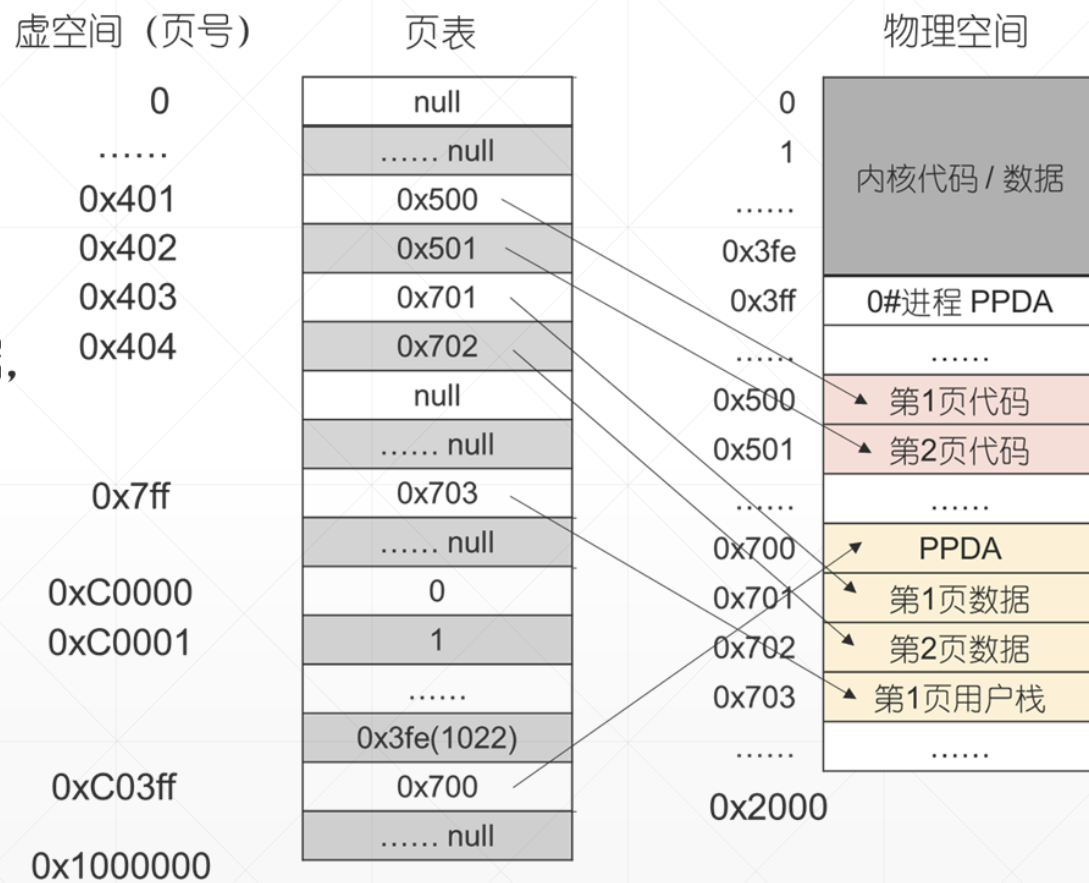
- 灵活。
- 以页为粒度，实现任何地址映射函数。

例1：Unix V6++风格的连续内存管理方式

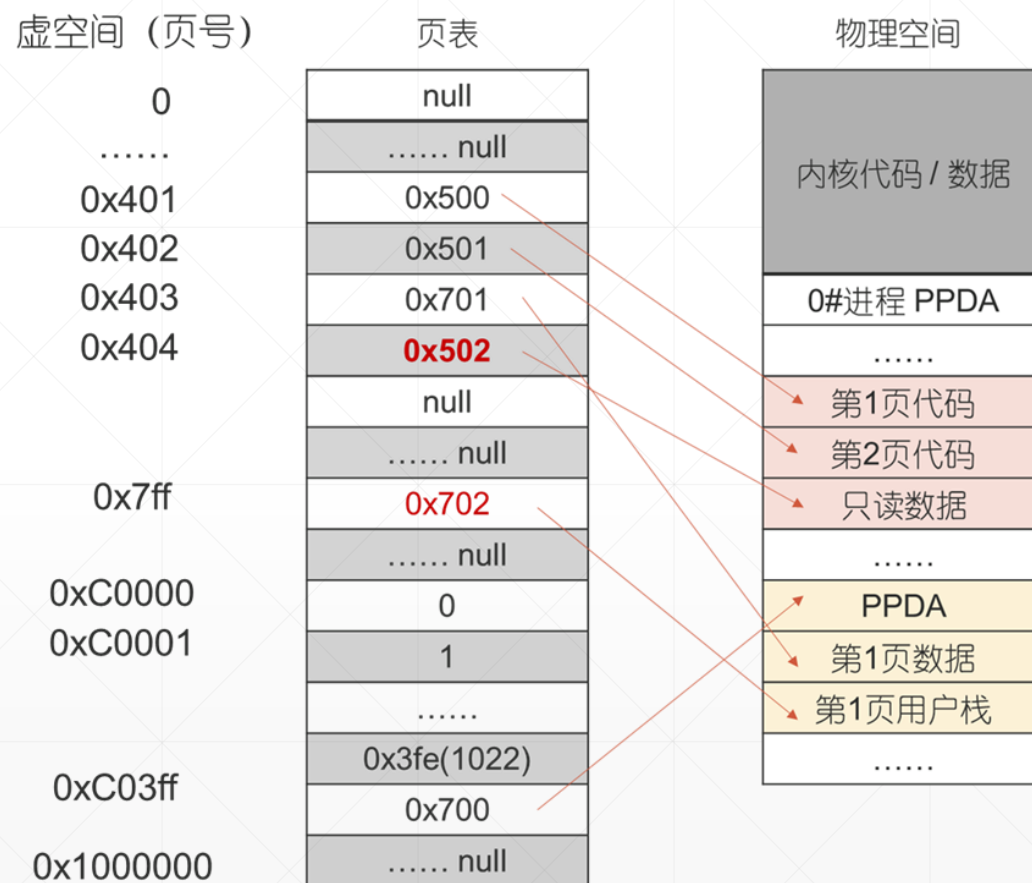


例2，把只读数据段放进共享正文段。 执行同一个应用程序的所有进程共用一个复本，减少进程图像的内存消耗

PC进程，
2页代码，
1页数据，
1页只读数据，
1页堆栈



合并前



合并后



页式MMU的第1个缺点

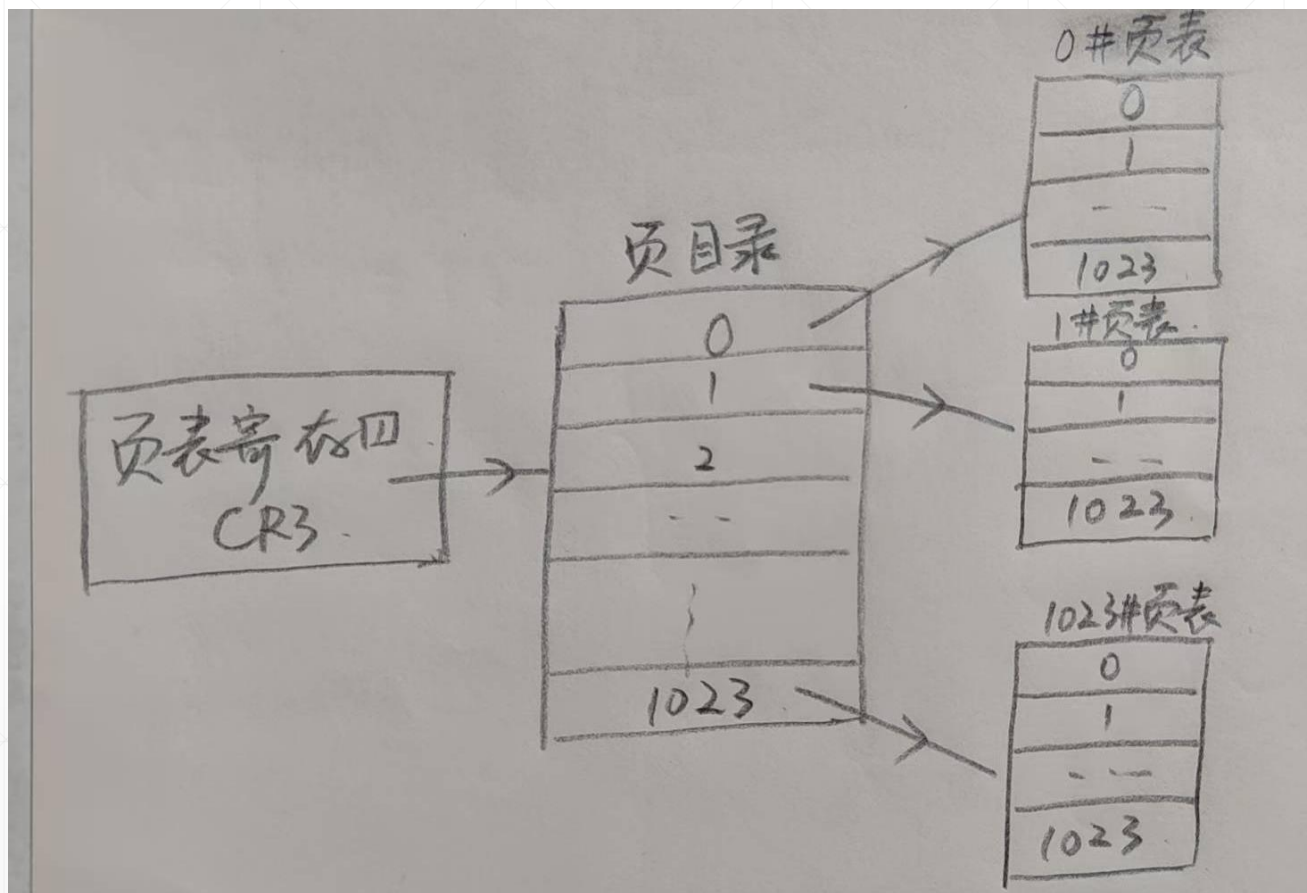
- 页表太大
- 描述4G空间，需要多少个页表项？
 $4G/4K = 1M$
- 每个页表项 4字节，页表需要消耗4M字节，连续物理内存。

虚空间 (页号)

页表

0	null
.....	 null
0x401	0x500
0x402	0x501
0x403	0x701
0x404	0x702
	null
	 null
0x7ff	0x703
	 null
0xC0000	0
0xC0001	1
	
	0x3fe(1022)
0xC03ff	0x700
0x100000	 null

i386使用二级页表，减小页表的内存消耗



4G字节虚空间，等分成1024份，每份4M字节。

内核为连续4M字节，分配一个页目录项和一张页表。若这4M字节空间，

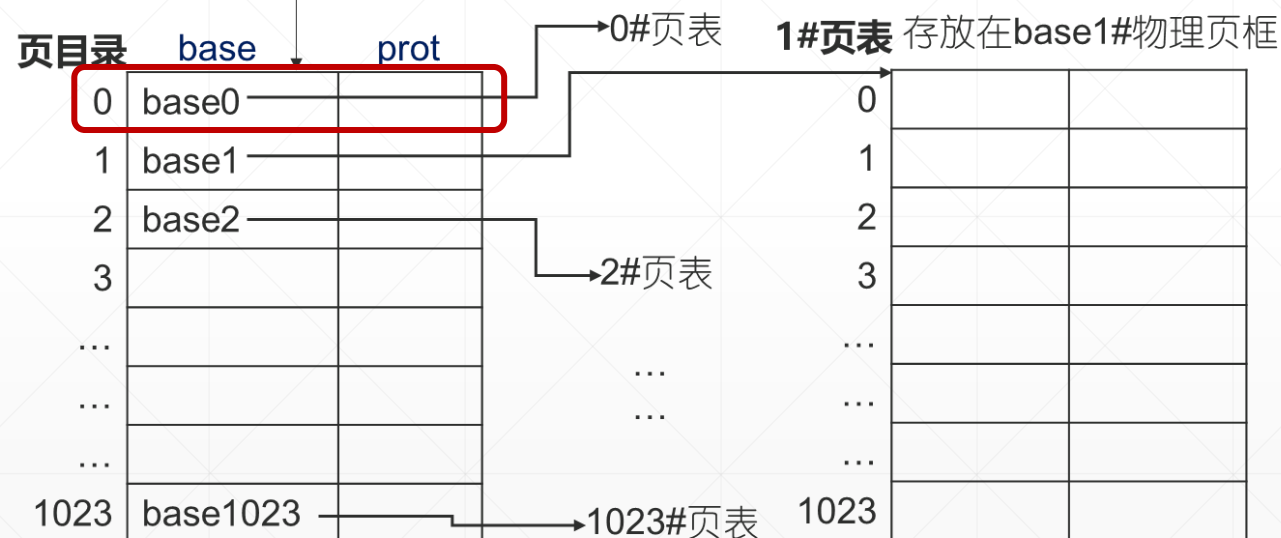
- 有代码数据，分配页表，记录虚实地址映射关系，页目录项登记分配给页表的物理页框号。
- 没有代码数据，无需分配页表，页目录项为空。

使用二级页表，无需为大片连续空白区域分配页表，可以有效降低页表项的内存消耗。

细节：i386使用的2级页表（1）



CR3引用一个页目录。页目录4096字节，存放在一个物理页框。CR3的值是页框的物理地址。

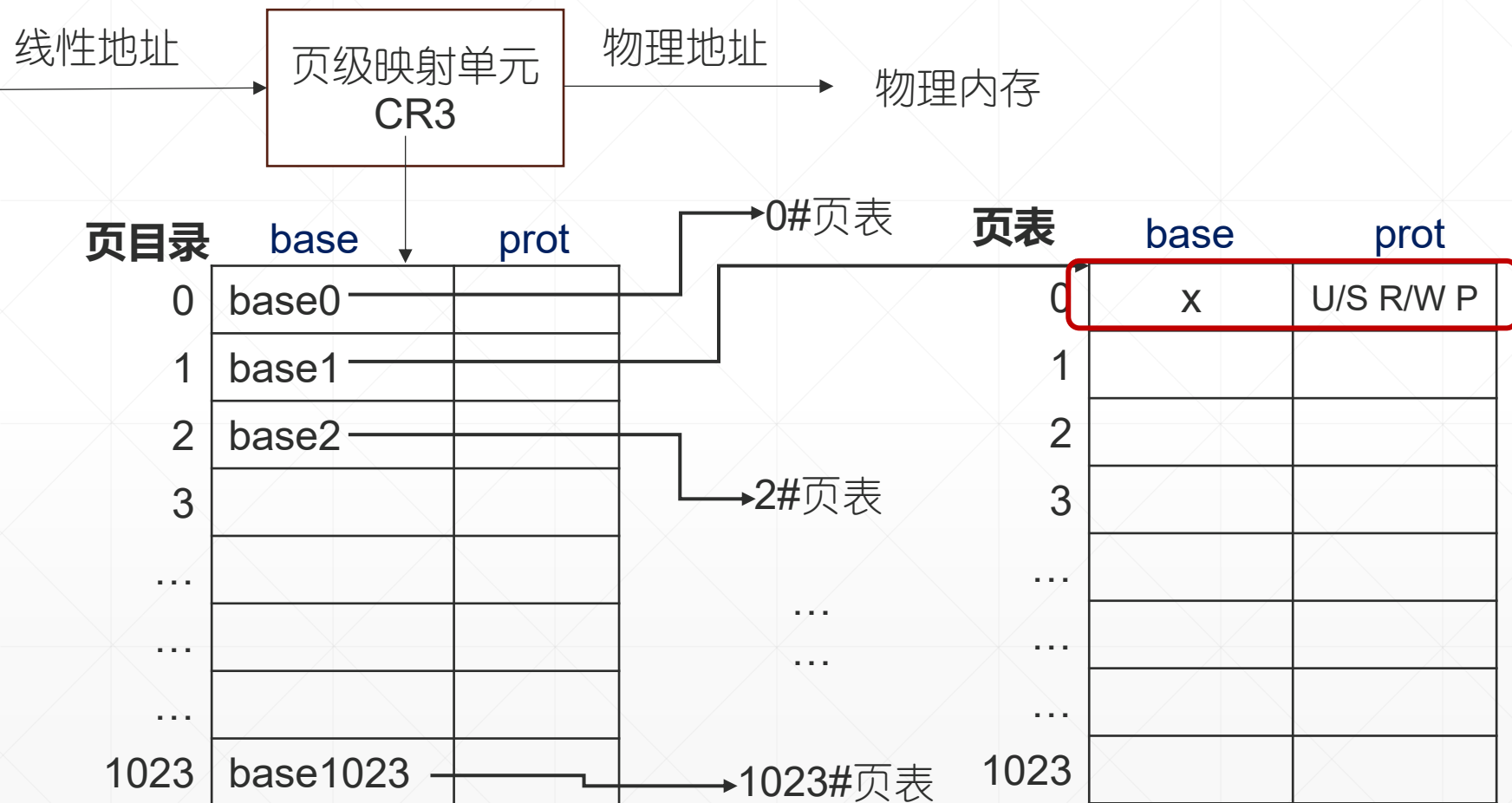


页目录项，4字节。页目录，1024个页目录项。虚空间4G字节， $4G/1K = 4M$ ，每个页目录项管理虚空间连续4M字节。

每个页目录项（PDE，Page Directory Entry），2个字段。base是一个物理页框号，prot是属性位，忽略。

物理页框base，存有一张页表，映射4M字节。页表1024项，每一项管理一个逻辑页，4096字节。

细节：i386使用的2级页表 (2)



每个页表项 (PTE, Page Table Entry), 管理一页。字段 **base** 是一个物理页框号, **prot** 是属性位。

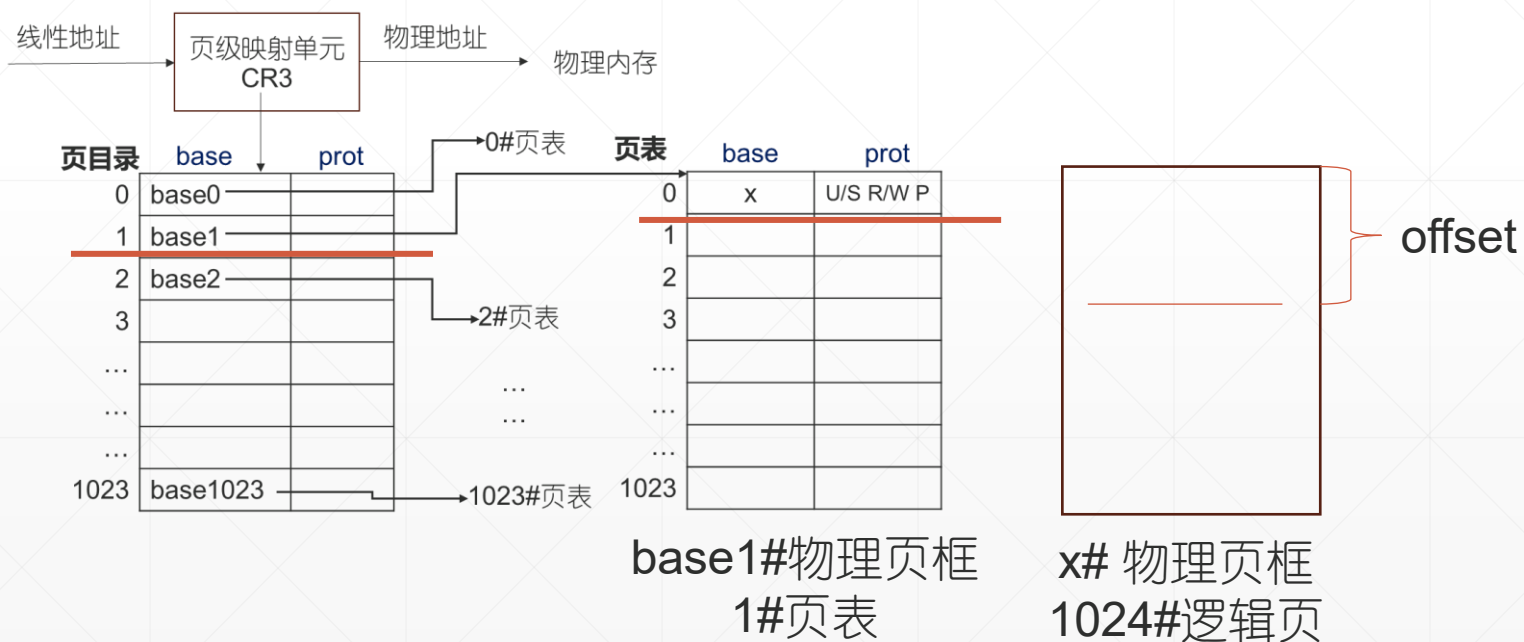
base# 物理页框, 存有一个逻辑页, 4096 字节。
prot 定义页面属性:
 U/S: 0 核心页, 1 用户页
 R/W: 0 只读, 1 可写
 P: 0 不存在, 1 存在

页级地址映射和内存保护 (1)



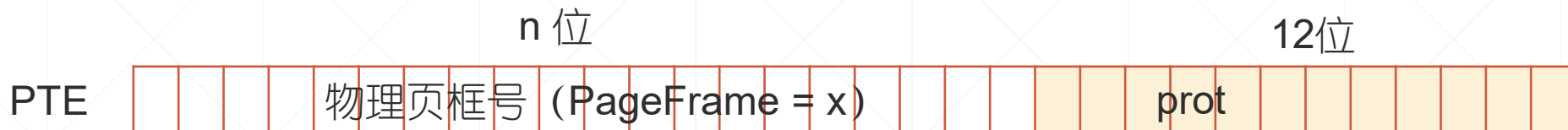
1、查页表

- 用页目录号查页目录
 - null, 非法内存访问
 - 非空, 映射需要的页表在 **base1#物理页框**
- 用页号查页表 (**base#物理页框**)
 - null, 非法内存访问
 - 非空, 获得映射所需的页表项





页级地址映射和内存保护 (2)



2、内存保护（检测prot中的bit）

CS

11

CPU用户态运行

if(CPU用户态运行 & U/S为0)

非法内存访问。应用程序无权访问内核，立即终止现运行进程。

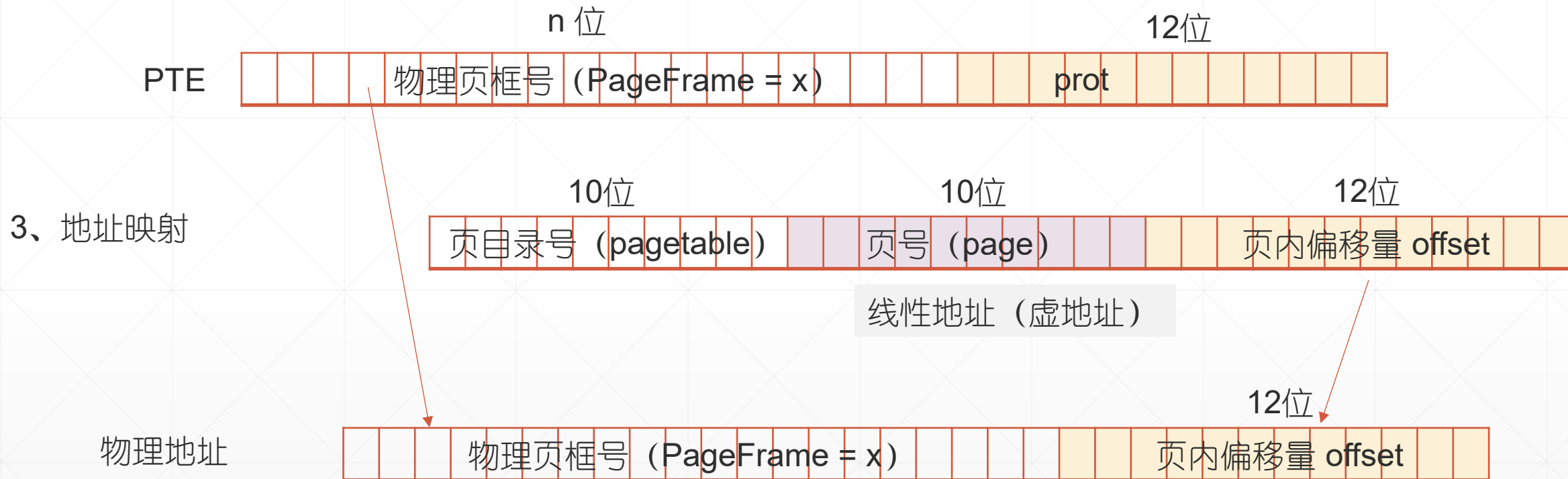
if(写内存单元 & R/W为0)

非法内存访问。访问的单元不可写，终止现运行进程。

if(CPU取指令 & 目标段X位是0, 不可执行)

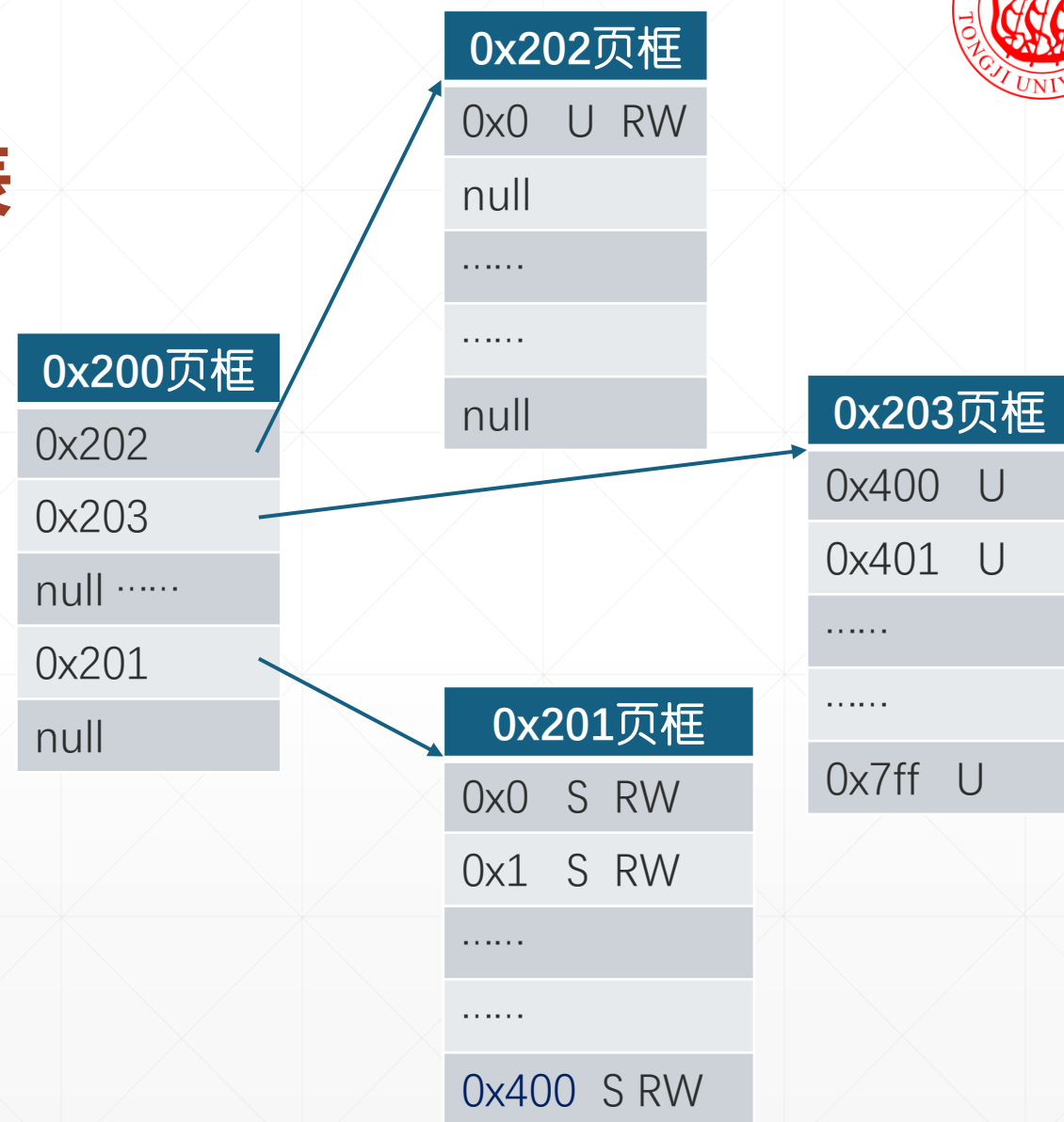
非法内存访问。修改代码段、只读数据段，终止现运行进程。

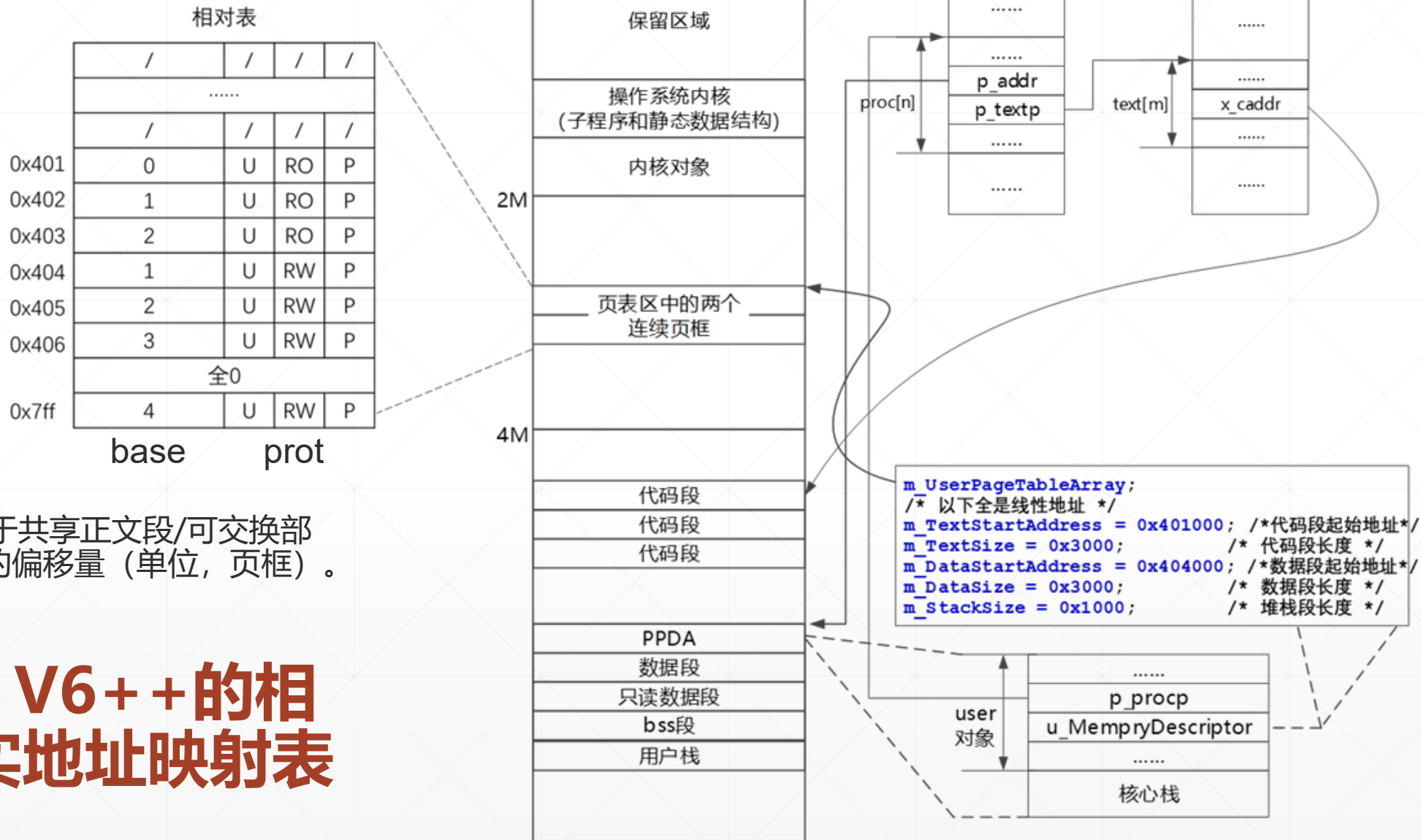
页级地址映射和内存保护 (3)



Unix V6++使用的二级页表

- 所有进程使用同一份二级页表，包括
 - 1张页目录，存放在0x200#物理页框
 - 1张核心页表，存放在0x201#物理页框
 - 2张用户页表，存放在0x202, 0x203#物理页框
- CR3的值 0x200000, 页目录的物理地址





base是相对于共享正文段/可交换部分起始页框的偏移量（单位，页框）。

Unix V6+ 的相对虚实地址映射表



选中目标进程后，为CPU建立新选中进程的地址映射关系

1、 $p_addr \gg 12$ ，填入核心页表1023#PTE的 base 字段，建立PPDA区的地址映射关系。

```
#define SwtchUStruct(p) \  
    Machine::Instance().GetKernelPageTable().m_Entrys[Kernel::USER_PAGE_INDEX].m_PageBaseAddress \  
        = (p)->p_addr / PageManager::PAGE_SIZE; \  
FlushPageDirectory();
```

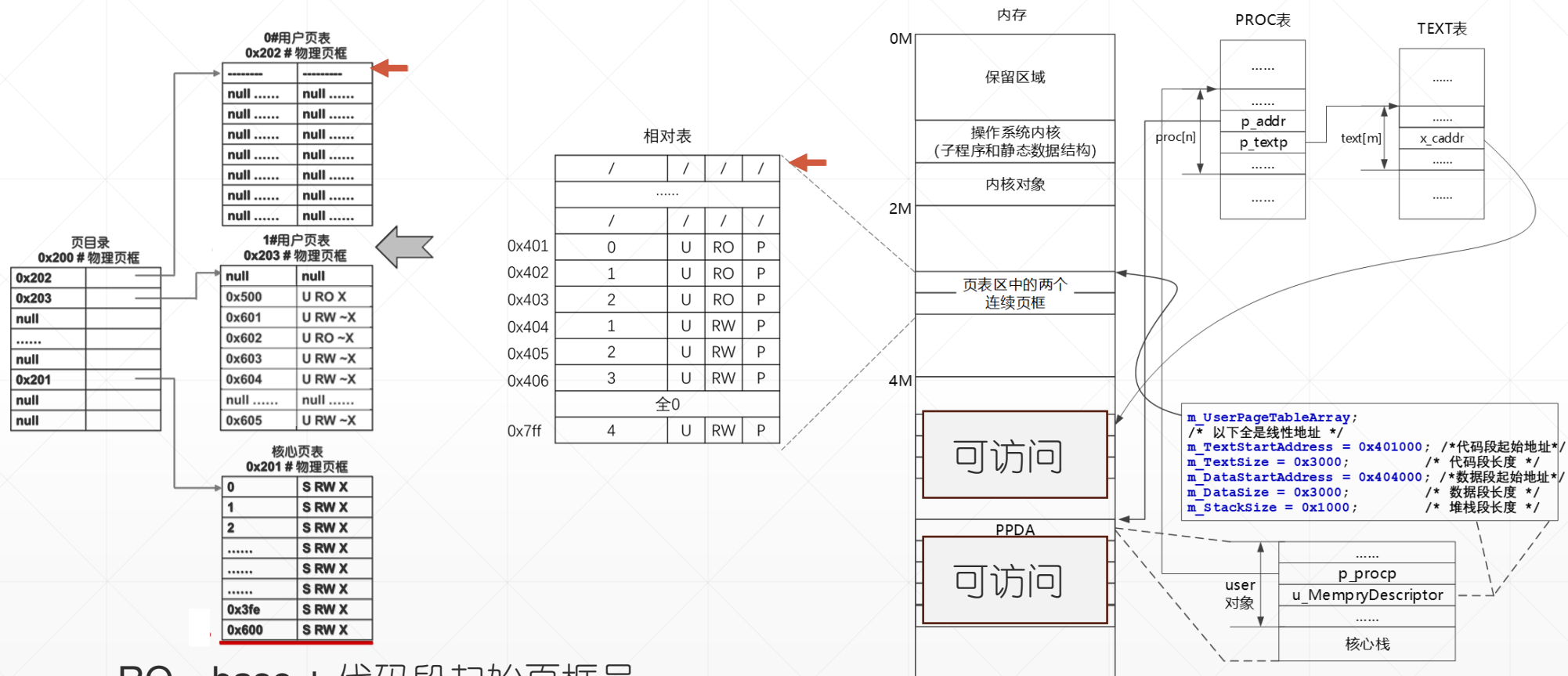
↑
PTE中的base

2、`User& u = Kernel::Instance().GetUser();`

访问user结构，得到相对表

3、刷新0x202、0x203#物理页框，建立新进程用户空间的地址映射关系

MapToPageTable()写系统页表



RO: base + 代码段起始页框号
RW: base + 可交换部分起始页框号



```
void MemoryDescriptor::MapToPageTable()
{
    User& u = Kernel::Instance().GetUser();

    if(u.u_MemoryDescriptor.m_UserPageTableArray == NULL)
        return;

    PageTable* pUserPageTable = Machine::Instance().GetUserPageTableArray();
    unsigned int textPF = 0;
    if ( u.u_procp->p_textp != NULL )
    {
        textPF = u.u_procp->p_textp->x_caddr >> 12;
    }

    unsigned int pAddrPF = u.u_procp->p_addr >> 12;
```



```
for (unsigned int i = 0; i < Machine::USER_PAGE_TABLE_CNT; i++)
{
    for ( unsigned int j = 0; j < PageTable::ENTRY_CNT_PER_PAGETABLE; j++ )
    {
        pUserPageTable[i].m_Entrys[j].m_Present = 0; // 清0表示该逻辑页不存在

        if ( 1 == this->m_UserPageTableArray[i].m_Entrys[j].m_Present )
        {
            if ( 0 == this->m_UserPageTableArray[i].m_Entrys[j].m_ReadWriter ) // RO逻辑页
            {
                pUserPageTable[i].m_Entrys[j].m_Present = 1;
                pUserPageTable[i].m_Entrys[j].m_ReadWriter = 0;
                pUserPageTable[i].m_Entrys[j].m_PageBaseAddress = this->
                    m_UserPageTableArray[i].m_Entrys[j].m_PageBaseAddress + textPF;
            }
            else if ( 1 == this->m_UserPageTableArray[i].m_Entrys[j].m_ReadWriter ) // RW逻辑页
            {
                pUserPageTable[i].m_Entrys[j].m_Present = 1;
                pUserPageTable[i].m_Entrys[j].m_ReadWriter = 1;
                pUserPageTable[i].m_Entrys[j].m_PageBaseAddress =
                    this->m_UserPageTableArray[i].m_Entrys[j].m_PageBaseAddress + pAddrPF;
            }
        }
    }
}
```



```
    pUserPageTable[0].m_Entrys[0].m_Present = 1;  
    pUserPageTable[0].m_Entrys[0].m_ReadWriter = 1;  
    pUserPageTable[0].m_Entrys[0].m_PageBaseAddress = 0;  
  
    FlushPageDirectory();  
}
```